Lab 4: Substitution Cipher

Information Security and Cryptography(AI331)

Objective

The objective of this lab is to understand and implement the **Substitution Cipher**, one of the fundamental symmetric ciphers.

Concept

A substitution cipher replaces each letter of the plaintext with another letter according to some fixed permutation of the alphabet.

Encryption Rule

Let f be a substitution function (a bijection on the English alphabet). For each plaintext character P_i, the ciphertext character is:

$$C_i = f(P_i).$$

Decryption Rule

The decryption process uses the inverse mapping f^{-1}:

$$P_i = f^{-1}(C_i).$$

Security Note

- Total number of possible keys is 26!, which is extremely large.
- However, substitution ciphers are easily broken using **frequency analysis**, since letter frequencies in English are not uniform (e.g., E, T, A, O are most common).
- Caesar cipher is a special case of substitution cipher (shift only).
- Affine cipher is another restricted form (linear mapping).

Lab Tasks

1. Generate a random substitution key (permutation of 26 letters).
2. Encrypt a given plaintext using the generated substitution key.
3. Decrypt the ciphertext using the inverse mapping.
4. (*Optional/Advanced*) Perform frequency analysis on ciphertext and compare with standard English frequencies.

Input/Output Format

- **Input file (plaintext.txt):**

```
Cryptography is fun and challenging!
```

- **Output:**
  - Print key mapping:

```
A -> Q
B -> W
...
Z -> M
```

  - Ciphertext written to `cipher.txt`.
  - Decrypted text written to `decrypted.txt`.

Skeleton Code (C++)

The following C++ code provides a structure for implementing the substitution cipher. Students are required to complete the missing parts.

```cpp
// Substitution Cipher Skeleton Code

#include <bits/stdc++.h>
using namespace std;

// Generate random substitution key
string generateKey() {
    string alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

```
    random_shuffle(alphabet.begin(), alphabet.end());
    return alphabet;
}

// Encrypt function (TO BE COMPLETED)
string encrypt(string text, string key) {
    // TODO: Implement encryption using the key mapping
    return "";
}

// Decrypt function (TO BE COMPLETED)
string decrypt(string cipher, string key) {
    // TODO: Implement decryption using the inverse key mapping
    return "";
}

int main() {
    string key = generateKey();
    cout << "Key: " << key << endl;

    string plaintext;
    cout << "Enter plaintext: ";
    getline(cin, plaintext);

    string cipher = encrypt(plaintext, key);
    cout << "Cipher: " << cipher << endl;

    string decrypted = decrypt(cipher, key);
    cout << "Decrypted: " << decrypted << endl;
}
```

Discussion Questions

1. How many possible substitution keys exist? Why is this theoretically secure?

2. Why is substitution cipher practically insecure despite its large key space?

3. How does frequency analysis help in breaking substitution ciphers?